

Julien HOARAU
Yann-Arthus RIBOUT

Rapport TP4

****1. Introduction****

Ce rapport présente le TP4 de programmation en C, consacré à la mise en œuvre d'un ****arbre binaire de recherche (ABR)**** pour gérer des mots et leur nombre d'occurrences. Les principales opérations implémentées sont l'ajout, le retrait et l'affichage de mots, la vérification de l'équilibre de l'arbre, ainsi que le calcul de la similarité entre deux arbres à l'aide de l'indice de Jaccard. Un menu interactif en mode texte permet de manipuler deux arbres (a1 et a2).

****2. Structures de données****

L'ensemble du projet repose sur la définition de plusieurs structures principales décrites dans le fichier d'en-tête.

* ****T_Noeud / T_Arbre**** : Chaque nœud contient un mot, le nombre de fois où il est apparu, et deux pointeurs vers ses fils gauche et droit. L'arbre est manipulé via un pointeur vers un nœud, ce qui permet une gestion dynamique.

* ****t_mot**** : Cette structure est utilisée pour représenter temporairement une liste chaînée de mots, notamment lors de parcours ou de conversions. Elle conserve le mot, son nombre d'occurrences, ainsi qu'un pointeur vers l'élément suivant.

* ****t_liste_arbre / t_pile_arbre**** : Ces structures permettent d'implémenter une pile de nœuds d'arbre, utile notamment dans les parcours itératifs ou les opérations nécessitant une mémorisation temporaire du chemin.

Ces structures permettent de manipuler dynamiquement les arbres binaires, tout en conservant un code clair, modulable et facilement évolutif.

****Ajouts et modifications du sujet****

* Deux fonctions supplémentaires, ``estEquilibre`` et ``trouverMin``, ont été implémentées pour analyser la structure des arbres et faciliter certaines opérations comme la suppression de nœuds.

Par rapport à l'énoncé initial, plusieurs ajouts ont été réalisés pour améliorer la gestion des ressources et faciliter certaines opérations :

* Une fonction ``toLowerCase`` a été ajoutée pour convertir les mots en minuscules avant leur traitement, assurant une comparaison uniforme.

* Deux fonctions de libération mémoire, ``detruireArbre`` et ``detruireListe``, permettent de désallouer proprement les structures dynamiques créées.

* Deux structures supplémentaires ont été introduites : ``t_pile_arbre``, utilisée pour les parcours infixes itératifs, et ``t_mot``, une liste chaînée servant de format intermédiaire pour l'affichage ou les comparaisons.

Ces extensions permettent une gestion plus robuste et modulaire du programme, tout en respectant les bonnes pratiques de programmation en C.

****3. Fonctionnalités implémentées****

* ****creerNoeud**** : cette fonction crée dynamiquement un nœud contenant un mot. Elle initialise les champs du nœud avec une occurrence à 1 et des pointeurs fils à NULL. Elle effectue une simple allocation et une copie de chaîne, donc sa complexité est constante : ****O(1)****.

* ****ajouterMot**** : cette fonction insère un mot dans l'arbre. Si le mot existe déjà, elle incrémente son nombre d'occurrences. Sinon, elle insère le mot à la bonne position pour respecter les règles d'un arbre binaire de recherche. La complexité dépend de la hauteur de l'arbre : ****O(h)****, soit ****O(log n)**** dans le meilleur des cas (arbre équilibré), et ****O(n)**** dans le pire des cas (arbre dégénéré).

* ****trouverMin**** : cette fonction renvoie le nœud contenant le mot minimal (le plus à gauche dans l'arbre). Elle descend récursivement ou itérativement vers le fils gauche jusqu'à atteindre une feuille. La complexité est ****O(h)****.

* ****retirerMot**** : cette fonction retire une occurrence d'un mot donné, ou le supprime complètement si son nombre d'occurrences devient nul. Elle traite trois cas : mot avec zéro, un ou deux fils. Lorsqu'un mot a deux fils, elle utilise ``trouverMin`` pour remplacer le mot supprimé par son successeur. La complexité de cette opération est ****O(h)****.

* ****afficherArbre**** : cette fonction effectue un parcours infixe (in-order) de l'arbre pour afficher les mots par ordre alphabétique. L'affichage est structuré selon la première lettre de chaque mot. Comme chaque nœud est visité une fois, la complexité est ****O(n)****.

* ****Fonction estEquilibre**** :

La fonction `estEquilibre` prend pour seul argument l'arbre dont on veut tester l'équilibre. Afin d'effectuer ce test, on utilise une fonction récursive qui a besoin d'un autre argument : la valeur de vérité du test.

La fonction `estEquilibreRec` est donc celle qui s'occupe réellement du test. Cette fonction ne renvoie pas la valeur de vérité mais un int permettant de stocker la hauteur des sous-arbres afin de ne pas devoir retraverser l'arbre à chaque calcul de hauteur nécessaire pour connaître l'équilibre.

Chaque appel de la fonction fait appel, dans le pire des cas, à 2 appels de la fonction et effectue un nombre constant d'opérations. Comme chaque nœud n'est parcouru qu'une seule fois, on a donc $2n$ appels où n est le nombre de nœuds de l'arbre. On a donc une complexité en ****O(n)****.

* **Fonction transformerArbre** :

La fonction transformerArbre prend pour seul argument l'arbre à transformer. Afin d'effectuer la transformation, on utilise une fonction récursive qui a besoin de deux autres arguments : la liste chaînée à renvoyer et la queue de ladite liste.

La fonction transformerArbreRec est donc celle qui s'occupe réellement de transformer l'arbre en liste. Pour ce faire, on parcourt l'arbre en infixe afin de traiter les nœuds dans l'ordre alphabétique.

Lors du traitement d'un nœud, il faut d'abord créer un mot correspondant puis l'ajouter à la liste. La création du mot est en temps constant ; quant à l'ajout à la liste, on peut simplement rajouter le nouveau mot en fin de liste puisque l'on traite les nœuds dans l'ordre alphabétique. Ainsi, cette étape et le traitement du nœud sont aussi en temps constant.

Le parcours infixe de l'arbre nécessite un passage par les nœuds ainsi que leur traitement, la complexité de transformerArbreRec est donc en $O(n)$ où n est le nombre de nœuds dans l'arbre.

La fonction transformerArbre ne faisant qu'un appel à transformerArbreRec et des opérations en temps constant est donc aussi en $O(n)$.

* **Fonction jaccard** :

Afin de calculer l'indice de Jaccard, nous avons choisi de parcourir en infixe les deux arbres de manière itérative en simultané. Pour cela, il a fallu implémenter deux fonctions auxiliaires : descenteGauche et depiler.

La fonction descenteGauche permet de placer le pointeur du parcours au nœud minimum d'un sous-arbre tout en empilant les nœuds traversés. Ceci correspond à la première étape du parcours infixe. Cette fonction a une complexité en $O(h)$ où h est la hauteur du sous-arbre appelé.

La fonction parcourir permet de passer d'un nœud à son successeur dans l'ordre du parcours infixe après son traitement. Pour cela, il y a deux cas possibles :

* Soit le nœud a un fils droit et on s'intéresse au sous-arbre droit en commençant par son nœud minimum que l'on trouve grâce à la fonction descenteGauche. Ce cas a une complexité en $O(h - 1)$ où h est la hauteur du nœud appelé.

* Sinon, il faut remonter dans l'arbre en dépilant la pile. Ce cas a une complexité en $O(1)$.

La complexité est donc en $O(h)$.

La fonction jaccard commence par initialiser les cardinalités de l'union et de l'intersection ainsi que les piles et les pointeurs.

4. Menu interactif

Le programme principal (`main`) propose une interface textuelle simple permettant à l'utilisateur d'interagir dynamiquement avec deux arbres binaires de recherche `a1` et `a2`. Il repose sur un menu à 7 options :

1. **Afficher un arbre** : Affiche les mots stockés dans un arbre en ordre alphabétique, regroupés par lettre initiale.
2. **Ajouter un mot dans un arbre** : Permet d'ajouter un mot à l'arbre sélectionné (a1 ou a2). Si le mot est déjà présent, son nombre d'occurrences est incrémenté.
3. **Retirer un mot d'un arbre** : Supprime une occurrence d'un mot. Si c'était la dernière, le nœud correspondant est retiré.
4. **Vérifier si un arbre est équilibré** : Utilise la fonction `estEquilibre` pour indiquer si l'arbre respecte les critères d'équilibre (différence de hauteur ≤ 1).
5. **Transformer un arbre en liste chaînée** : Convertit l'arbre sélectionné en une liste chaînée ordonnée, stockée dans `l1` ou `l2`.
6. **Calculer l'indice de Jaccard** : Affiche la similarité entre les deux arbres, en tenant compte des mots communs et distincts.
7. **Quitter** : Libère la mémoire allouée aux arbres et aux listes, puis termine le programme.

Chaque choix est validé par un `scanf` et suivi d'un appel aux fonctions correspondantes, assurant une interaction fluide et modulaire avec les structures de données manipulées.